

Документация проекта Fraud Detection Service

1 Описание структуры проекта

```
fraud-detection-service/  
  
    src/  
        app.py  
        main.py  
        cli.py  
        config.py  
        preprocessing.py  
        inference.py  
        explainability.py  
        gan.py  
  
    artifacts/  
  
    data/  
  
    tests/  
        test_api.py  
        test_preprocessing.py  
        conftest.py  
  
    Dockerfile.backend  
    Dockerfile.frontend  
    docker-compose.yml  
    requirements.txt  
    pyproject.toml  
    poetry.lock  
    artifacts.dvc  
    data.dvc  
    dvc.yaml  
    README.md
```

2 Описание файлов

2.1 Папка src

- `app.py` — Streamlit frontend;

- `main.py` — FastAPI backend;
- `cli.py` — CLI-интерфейс для работы с сервисом;
- `config.py` — конфигурация проекта;
- `preprocessing.py` — preprocessing pipeline;
- `inference.py` — inference ML-моделей;
- `explainability.py` — SHAP explainability;
- `gan.py` — вычисление GAN anomaly score.

2.2 Папка `artifacts`

Содержит модели, preprocessing artifacts и metadata.

2.3 Папка `data`

Содержит parquet datasets и sample transactions.

2.4 Папка `tests`

Содержит smoke tests для API и preprocessing pipeline.

3 Описание функционала

3.1 FastAPI сервис

3.1.1 `/health`

- Метод: GET
- Назначение: проверка состояния backend-сервиса

3.1.2 `/models`

- Метод: GET
- Назначение: получение списка доступных моделей

3.1.3 `/model_info`

- Метод: GET
- Назначение: получение информации о моделях и preprocessing pipeline

3.1.4 /predict

- Метод: POST
- Назначение: inference ML-моделей
- Входные параметры:
 - model_name
 - transaction
- Возвращает:
 - fraud probability
 - prediction label
 - risk level
 - SHAP explanation

4 Preprocessing pipeline

Перед inference выполняется preprocessing входных данных:

- обработка пропусков;
- приведение типов;
- feature engineering;
- заполнение отсутствующих признаков;
- приведение признаков к train-time feature order;
- формирование итогового feature vector.

5 ML-модели

5.1 LightGBM

Основная supervised-модель для классификации мошеннических транзакций.

5.2 Hybrid LightGBM + GAN

Hybrid pipeline состоит из двух этапов:

1. GAN discriminator вычисляет anomaly score;
2. anomaly score добавляется как дополнительный признак в LightGBM.

6 SHAP Explainability

Для explainability используется SHAP.

Сервис отображает:

- top feature impacts;
- SHAP values;
- наиболее значимые признаки;
- влияние признаков на fraud prediction.

7 Streamlit приложение

7.1 Вкладка Prediction

- загрузка CSV-файла;
- preview transactions dataframe;
- выбор модели;
- выбор транзакции;
- prediction fraud probability;
- отображение risk level;
- SHAP explainability;
- отображение наиболее рискованных транзакций.

7.2 Вкладка Model info

- отображение списка моделей;
- отображение preprocessing metadata;
- отображение информации о feature space;
- отображение model versions.

8 CLI интерфейс

Для взаимодействия с сервисом без запуска Streamlit/FastAPI реализован CLI-интерфейс на базе argparse.

CLI позволяет:

- получать информацию о сервисе и моделях;
- выполнять prediction для отдельной транзакции;
- выполнять batch prediction для CSV-файлов.

8.1 Доступные команды

8.1.1 Информация о сервисе

```
python -m src.cli info
```

8.1.2 Prediction для одной транзакции

```
python -m src.cli predict --input data/sample_transactions.csv --  
row 0 --model lgbm
```

8.1.3 Batch prediction

```
python -m src.cli batch-predict --input data/sample_transactions.  
csv --output data/predictions.csv --model lgbm
```

9 Docker инфраструктура

Проект разворачивается через Docker Compose.

9.1 Сервисы

- fraud-api — FastAPI backend;
- fraud-ui — Streamlit frontend.

10 Poetry и управление зависимостями

Для управления Python-зависимостями используется Poetry.

Poetry используется для:

- фиксации зависимостей проекта;
- создания воспроизводимого окружения;
- управления virtual environment;
- запуска CLI и тестов.

Основные команды:

```
poetry install
```

```
poetry run pytest
```

```
poetry run python -m src.cli info
```

11 DVC и хранение артефактов

Для хранения моделей и данных используется DVC.

В качестве remote storage используется Yandex Object Storage.

DVC используется для:

- хранения ML-моделей;
- хранения preprocessing artifacts;
- хранения feature metadata;
- хранения parquet datasets;
- воспроизводимости ML pipeline;
- автоматического восстановления артефактов при разворачивании сервиса.

Артефакты и данные автоматически загружаются через:

```
poetry run dvc pull
```

12 Инструкция по запуску

12.1 Требования

- Docker Desktop
- Python 3.10+
- Poetry
- рекомендуется 8 GB RAM

12.2 Клонирование репозитория

```
git clone https://github.com/realtonysoprano/fraud-detection-  
service.git  
cd fraud-detection-service
```

12.3 Установка зависимостей

```
poetry install
```

12.4 Загрузка моделей и данных через DVC

```
poetry run dvc pull
```

12.5 Запуск проекта

```
docker compose up --build
```

12.6 Доступ к сервисам

Streamlit UI:

```
http://localhost:8501
```

FastAPI Swagger Docs:

```
http://localhost:8000/docs
```

13 Инструкция по использованию

1. Открыть Streamlit UI;
2. Загрузить CSV-файл (или использовать по умолчанию загруженные данные);
3. Выбрать модель;
4. Выбрать транзакцию;
5. Выполнить prediction;
6. Просмотреть fraud probability и SHAP explanation.

13.1 Использование CLI

Примеры CLI-команд:

```
python -m src.cli info
```

```
python -m src.cli predict --input data/sample_transactions.csv --  
row 0 --model lgbm
```

```
python -m src.cli batch-predict --input data/sample_transactions.  
csv --output data/predictions.csv --model lgbm
```

14 Тестирование

Для тестирования используется pytest.

```
poetry run pytest
```

Тестируются:

- preprocessing pipeline;
- FastAPI endpoints;
- корректность response structure.